Post-training

Lecture 3: how a base model becomes an assistant

Aarne Talman

3 November 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

Course overview

Wk	Dates	Tuesday	Thursday	Teacher
1	27 / 29 Oct	Language models	Pretraining	Aarne
2	**3 / 5 Nov**	**Post-training**	**Lab 2: tune vs prompt**	**Aarne**
3	10 / 12 Nov	Inference	Lab 3: serve	Aarne
4	17 / 19 Nov	Evaluation	Lab 4: evaluate	Jussi
5	24 / 26 Nov	RAG	Lab 5: RAG	Dmitry
6	1 / 3 Dec	Agents	Lab 6: agent	Dmitry
7	8 / 10 Dec	Course recap	Special topics (TBC)	All

Reusing a task and model across labs is recommended, not required.

There are no intermediate submissions; the final report and code archive determine the grade.

Lecture overview

1. Base vs instruct *10 min*
2. The post-training pipeline *5 min*
3. Supervised fine-tuning *15 min*
4. Preference optimisation *15 min*
5. RL with verifiable rewards: GRPO *15 min*
6. Parameter-efficient training *10 min*
7. Prompt, retrieve, or fine-tune? *10 min*
8. Lab 2 briefing *10 min*

Take away

By the end of today you should be able to explain how supervised fine-tuning, preference optimisation and verifiable rewards change a base model, and choose a practical method for a small project.

Base vs instruct

The same prompt, two very different replies

Base model

```
Give me two numbers that sum
to 10 and multiply to 21.

Give me two numbers that sum
to 12 and multiply to 35.
Give me three numbers that
sum to 9 and multiply to 24.
Solutions are on page 212.
```

Instruct model

```
Give me two numbers that sum
to 10 and multiply to 21.

3 and 7.

(3 + 7 = 10, and 3 x 7 = 21.)
```

The architecture, the tokenizer and the pretraining data are identical in both columns. What differs is **post-training**, plus the interface wrapped around it.

Why a base model may continue the document

A base model was trained to continue documents, not specifically to answer a user.

- It has no notion of a *turn*, so it does not know whose text comes next
- It has no notion of *being asked*, so a question is just text that questions tend to follow
- It has no reason to *stop*, because documents do not stop after one paragraph

In the training corpus, a line like that one usually appears on a worksheet. So it writes the rest of the worksheet.

What post-training changes

Much of the model's general knowledge comes from pretraining.

Post-training mainly changes which continuation the model considers likely, once it is given a structure that says *a user asked, now the assistant answers, and then it stops*.

That structure is a **string format**, and the model learns it the same way it learned everything else.

A CHAT IS A STRING

`apply_chat_template` takes a list of role/content dictionaries and returns one flat string. Everything the model ever sees is that string.

```
msgs = [{"role": "system",  "content": "You are a helpful assistant."},
        {"role": "user",    "content": "Give me two numbers that sum to 10 and multiply to 21."}]
tok.apply_chat_template(msgs, tokenize=False, add_generation_prompt=True)
```

returns, verbatim — newlines and all:

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
Give me two numbers that sum to 10 and multiply to 21.<|im_end|>
<|im_start|>assistant
<think>

</think>
```

The template is a Jinja program shipped with the weights

```
{%- for message in messages %}
    {{- '<|im_start|>' + message.role + '\n' + message.content + '<|im_end|>' + '\n' }}
{%- endfor %}
{%- if add_generation_prompt %}
    {{- '<|im_start|>assistant\n' }}
    {%- if not enable_thinking %}
        {{- '<think>\n\n</think>\n\n' }}
    {%- endif %}
{%- endif %}
```

It lives in `tokenizer_config.json`, under `chat_template`. It is **part of the model** and **different for every model family**, even though it is nowhere in the `.safetensors` file.

Read it before you fine-tune. It tells you where the assistant region begins, which is where your loss mask has to go.

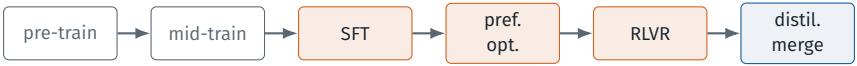
Anatomy of the interface

Piece	What it is
roles	`system`, `user`, `assistant`, and in 2026 almost always `tool`. Pure convention, learned during SFT
special tokens	real vocabulary entries (<|`im_start`|>, <|`im_end`|>), usually reserved during pretraining and given meaning afterwards
generation prompt	the trailing *open* assistant turn. Without it the model has no reason to think it is its turn to speak
stop condition	the end-of-turn token, which the model is trained to emit and the serving stack is configured to stop on
system prompt	one more turn, first in the string. Its only power is the position the model was trained to respect

A base model has no `chat_template` field at all. Apply one anyway and you are feeding it tokens it has never seen in a meaningful context, so your baseline will be wrong.

The post-training pipeline

The 2026 pipeline, and what each stage fixes



SFT	supervised fine-tuning: teaches the format and *a* good answer. Cannot teach “better than”, and needs written responses
pref. opt.	teaches “better than” from pairwise human judgement. Bottlenecked by preference labels
RLVR	reinforcement learning with verifiable rewards: removes the human wherever the answer can be checked by a program, so it scales with compute rather than with the number of annotators
distil.	recovers what the RL stages damaged, and collapses many specialists into one shippable checkpoint

The pipeline forgets its own earlier stages

GLM-5's final distillation step exists, in the authors' own words, *to recover capabilities that may have weakened during the sequential RL stages.*

That is **catastrophic forgetting inside the post-training pipeline itself**. Four RL stages aimed at reasoning, agents and general behaviour each degraded something the earlier stages had installed, and the fix was to distil from the checkpoints that still had it.

You will run into the same thing on Thursday with a single GPU.

Take away

Every post-training stage is a trade. Asking whether it improved the target metric is not enough; you also have to ask what it cost somewhere else.

Lab 2 measures exactly this, with a fifty-question general-knowledge check before and after your fine-tune.

Source: Raschka, *Controlling Reasoning Effort in LLMs*, July 2026, quoting the GLM-5 technical report.

Supervised fine-tuning

SFT IS PRETRAINING WITH BETTER MANNERS

$$\mathcal{L}_{\mathrm{SFT}} \;=\; -\sum_{t=1}^{T} m_t \,\log p_\theta(x_t \mid x_{<t}) \qquad \text{with } m_t \in \{0,1\}$$

In words: the week 1 next-token loss, with an on/off switch m_t in front of every token. Switch it off and that token is read but not learned.

Compared with week 1, exactly three things change:

1. the **data** is conversations rendered through the model's own chat template, not web documents
2. the **mask** m_t zeroes the loss on everything that is not assistant content
3. the **budget** is tiny: thousands of examples and one GPU, against trillions of tokens and a cluster

Nothing happens to the optimiser, the objective or the architecture. That is what makes SFT cheap, and it is also the reason it cannot do very much.

The loss mask, drawn



Abbreviations: `im_s` is <|`im_start`|>, `im_e` is <|`im_end`|>, `asst` is `assistant`.

All of it is still *in the context*, and the model attends to all of it. Only the blue cells contribute a gradient. Notice that the final end-of-turn token sits **inside** the mask, because stopping is a behaviour you have to train.

What goes wrong if you do not mask

Train on everything and the model learns to be the user too.

- it will cheerfully generate the next *user* turn after finishing its own
- the gradient is spent reproducing text you already have
- with long prompts and short answers (classification, NER, extractive QA) most of your compute goes on memorising inputs
- the loss curve looks beautiful, because predicting your own prompts is easy

In practice, on Thursday

TRL's `SFTTrainer` will do this for you, but only if you ask: `assistant_only_loss=True`.

That flag requires the chat template to mark the assistant region. If the template does not, you get the whole sequence in the loss and **no warning**.

Check it by printing the labels of one batch and confirming that the ignored positions are exactly the prompt.

Data: a thousand good examples go a long way

The argument, in one line: SFT **selects** a behaviour the model already has, and to do that you need clean, consistent demonstrations rather than a lot of them.

- LIMA fine-tuned a 65B model on **1,000** curated examples and remained competitive with far larger instruction sets [Zhou et al. 2023]
- one inconsistent example teaches the model that the format is optional
- a scraped set teaches you the average of its annotators, including their worst day

What "quality" concretely means

consistent format — the same answer shape every time

correct — you checked, with a script where possible

on-distribution — the prompts look like the prompts you will receive in practice

diverse — across the input space, not across phrasings of the same input

deduplicated — against each other and against your test set

For Lab 2, **1–5k examples** is the right order of magnitude. You will get more out of an afternoon cleaning them than out of a week collecting more.

Catastrophic forgetting

What it is. Learning new behaviour moves weights, and the behaviour those weights previously encoded degrades. In continued pretraining this is well documented; in SFT it is the same mechanism with a smaller step size.

Why it is unavoidable. There is no free capacity lying around. A gradient computed on a narrow distribution overwrites a representation that the rest of the model was sharing.

What makes it worse: high learning rate, many epochs, narrow data, full fine-tuning rather than LoRA, and a small model.

What it looks like

Your task metric goes up by 12 points.

The model now answers every question in your task's format, including "what is the capital of Finland".

It has stopped using the second language it used to handle.

It no longer refuses anything.

None of this appears in your task metric.

Mitigations, in increasing cost: fewer epochs • lower learning rate • LoRA with a small rank • mixing general data back in (rehearsal) • distilling from your own pre-SFT checkpoint.

Measure what broke, not only what improved

Use a multiple-choice benchmark such as **MMLU** as a **regression test**, not as a capability score.

- score the model you started from, score the fine-tuned model, report the *difference*
- the absolute number tells you very little. The change tells you a lot
- a high score is weak evidence that the model is good, but a drop is strong evidence that you broke something

Random guessing on four options is **25%**, so anything near that is a scoring bug, not a result.

Take away

In Lab 2 this is the fourth column of your results table: fifty general-knowledge questions, scored *before* and *after*, with identical decoding settings.

A report with three numbers and no forgetting check is incomplete.

Source: Raschka, *Understanding the 4 Main Approaches to LLM Evaluation*: multiple-choice benchmarks are most useful for "checking if and how much knowledge the model has forgotten compared to the base model".

Preference optimisation

Why SFT is not enough

SFT maximises the likelihood of **one** demonstrated answer. It has no way to express that one answer is **better than** another.

That matters whenever the target is a *judgement* rather than a fact:

- helpfulness, tone, length, hedging
- refusing the right things and only those
- "answer the question that was asked"
- formatting that nobody can write down as a rule

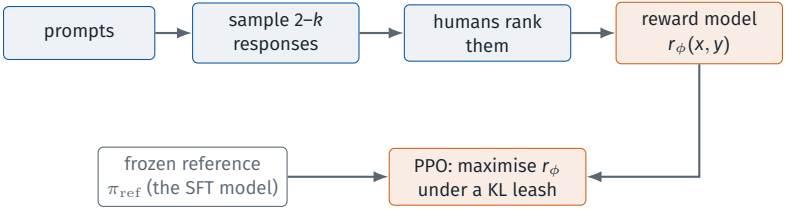
Writing the gold answer for any of these is hard, while *choosing* between two candidates is easy, and every method in this section is built on that gap.

The second problem SFT has

SFT never sees a bad answer. It is trained only on positives, so it has no gradient that pushes probability mass *away* from the failure modes it will actually produce.

Every method in this section is, mechanically, a way to add that negative signal.

RLHF IN ONE DIAGRAM



The RL vocabulary, once, in plain words. The **policy** is the model being trained. The **reference** is a frozen copy of it from before this stage. The **KL leash** is a penalty for drifting away from that copy. The **value model** (or critic) is a second network whose only job is to guess how good an answer will turn out.

That is four large models in memory at once, which is why the field went looking for something simpler.

Source: Ouyang et al. 2022, *Training language models to follow instructions with human feedback* (InstructGPT).

The reward model is logistic regression on pairs

Collect triples: a prompt x, the answer a human *preferred* (y_w) and the one they *rejected* (y_l). Train a scorer r_ϕ so that

$$P(y_w \text{ wins}) \;=\; \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

In words: the bigger the score gap between two answers, the more confidently we predict which one a human picked. You fit it like any logistic regression, by maximising the probability it assigns to the choices people made. The recipe has a name, **Bradley–Terry**, and it returns in week 4 for arena leaderboards.

r_ϕ is usually the SFT model with its next-token head swapped for a head that emits a single number. Only *differences* of scores are pinned down, because the scale has no absolute zero.

The RL stage then looks for the model that earns the highest average score from r_ϕ, minus a penalty β times how far it has drifted from the frozen starting model.

PPO, briefly, because GRPO is a simplification of it

$$\mathcal{J}_{\text{PPO}} = \mathbb{E}\Big[\min\Big(\rho\hat{A},\ \text{clip}(\rho)\,\hat{A}\Big)\Big], \qquad \rho = \frac{\pi_{\text{new}}(\text{this token})}{\pi_{\text{old}}(\text{this token})}$$

In words: make the tokens of better-than-average answers more likely and the rest less likely, with a cap on how far one batch may move the model.

Advantage $\hat{A}$ — how much better than average this answer turned out to be. "Average" has to come from somewhere.

Clipping — the cap. The ratio is held within about 20% of 1. Without it, one bad batch wrecks the model.

The critic — a second network, as large as the one being trained, whose job is to predict that average.

The part that matters next

The critic exists only to supply that "average", and it is another model the size of the one being trained, with its own loss and its own failure modes.

GRPO's contribution is a cheaper way to get the same average.

Source: Schulman et al. 2017, *Proximal Policy Optimization Algorithms.*

Reward source and optimisation are separate choices

	Reward source	Reward model?	Critic?
RLHF + PPO	learned human-preference score	yes	yes
GRPO	learned score	yes	**no**
RLVR + GRPO	programmatic verifier	**no**	**no**

- **GRPO removes the critic**: it compares several responses to the same prompt instead of predicting a baseline with another network.
- **RLVR changes the reward source**: a program checks the answer. GRPO can also be used with a learned reward.
- **DPO takes a different route**: it trains on stored preference pairs, without generating responses during the update.

DPO LEARNS DIRECTLY FROM PREFERRED AND REJECTED ANSWERS

A DPO example contains a prompt, a **preferred answer** and a **rejected answer**. Training increases the preferred answer's likelihood relative to the rejected one.

Why it is useful

- people often find comparison easier than writing an ideal answer;
- training resembles ordinary supervised learning;
- there is no separate reward model or rollout loop.

What to watch

- the pair only tells us which answer is better;
- the data may reflect annotator or judge bias;
- a frozen reference model keeps the update close to its starting behaviour.

Take away

Use DPO when you have reliable preference pairs and the target quality is difficult to express as an automatic score.

Source: Rafailov et al. 2023, *Direct Preference Optimization: Your Language Model is Secretly a Reward Model.*

Where preference machinery survives in 2026

It did not die. What happened is that it moved over to the **non-verifiable half** of the reward.

DeepSeek V3.2, verbatim: rule-based outcome reward, length penalty and language-consistency reward *for reasoning and agent tasks*; a **generative reward model where each prompt has its own rubrics** *for general tasks*.

So the 2026 pipeline is a **hybrid**: verifiers where the answer can be checked, LLM-as-a-judge with rubrics everywhere else. **Kimi K3** likewise keeps learned reward models and pairwise comparisons for its non-verifiable and agentic tasks.

Take away

A clean diagnostic, usable on your own project:

strong on multiple-choice and verifier benchmarks but weak with an LLM judge or on an arena ⇒ the model knows the answer and cannot say it well ⇒ this is the one case where preference optimisation is the right tool.

Sources: Raschka, *From DeepSeek V3 to V3.2*, December 2025; *Understanding the 4 Main Approaches to LLM Evaluation.* Model facts as of October 2026.

RL WITH VERIFIABLE REWARDS: GRPO

The observation that produced reasoning models

In one sentence

If you can **check** the answer automatically, you do not need to model human preference at all, which removes the part of post-training that refused to scale.

SFT needs written responses and RLHF needs preference labels. You buy both from people, at a fixed price per item.

A verifier removes most per-item human labelling, but generation and checking still cost compute. Post-training therefore shifts much of the budget from annotation to computation.

Names and dates

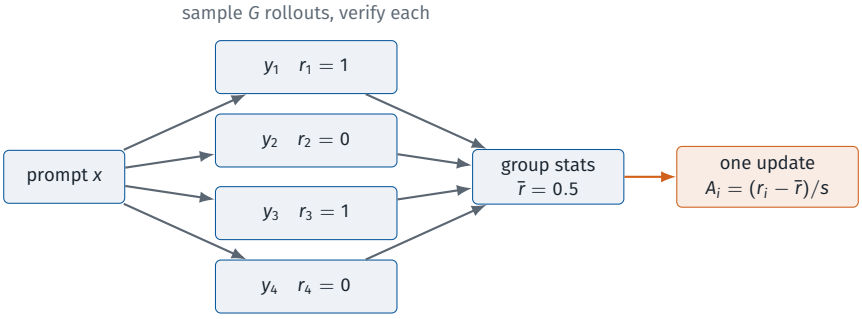
RLVR coined in **Tülu 3**, roughly two months before DeepSeek-R1.

DeepSeek-R1 and **Kimi K1.5** were posted to arXiv on the same day, **22 January 2025**.

R1 became the famous one because of a single result: reasoning behaviour from *pure* RL.

Source: Raschka, *Build Your Own Reasoning Model*, October 2026; *The State of LLMs 2025*.

GRPO: ONE PROMPT, A GROUP OF ROLLOUTS, ONE UPDATE



1. Sample G complete responses for the *same* prompt ("rollout" is jargon for the full generated sequence). **2.** Verify each: 1 if correct, 0 if not. **3.** Score each response relative to the rest of *its own group*. **4.** Clipped policy-gradient update.

Source: Raschka, *Build Your Own Reasoning Model*, October 2026, videos 6–7.

GRPO USES THE GROUP AS ITS BASELINE

$$A_i = \frac{r_i - \text{mean reward of the group}}{\text{standard deviation of the group}}$$

In words: the **advantage** says how much better an answer was than the other answers to the same prompt.

- PPO trains a critic to predict this baseline. GRPO measures it from the group, so there is no critic to hold or train.
- Example: rewards $1, 0, 1, 0$ give advantages $+0.87, -0.87, +0.87, -0.87$.
- If every answer receives the same reward, every advantage is zero and that group teaches nothing.

DeepSeek-R1-Zero: no SFT at all

R1-Zero applied RLVR **directly to the pretrained base model**. No supervised fine-tuning, no demonstrations, no preference data.

The reward was two things added together, **is the answer right** and **is it laid out correctly**, plus a language-consistency term to stop the model alternating between languages mid-answer.

R1 itself is the practical model, trained in several stages from an SFT checkpoint of DeepSeek-V3. R1-Zero was the weaker of the two and by far the more important.

What each reward does

accuracy — is the final boxed answer right? One or zero, from a symbolic checker or a compiler.

format — is the reasoning inside the thinking tags and the answer where the parser expects it? This is the *only* reason the tags exist.

language consistency — a patch for a real failure, where the model drifted between languages because nothing penalised it.

Sources: Raschka, *Build Your Own Reasoning Model*, October 2026; *From DeepSeek V3 to V3.2*, December 2025. DeepSeek-R1 paper, January 2025.

The counter-intuitive result

The reasoning trace is not directly supervised

In R1-Zero, the reward depends on the **final answer** and the **format**. The intermediate reasoning is not scored, not supervised, and not used to update the model. DeepSeek tried using it and reported that it did not help.

Backtracking, self-checking, trying a second approach and abandoning a failed one all **emerge anyway**, from outcome rewards alone.

Why it works. Traces that happen to contain a self-correction reach the right answer more often. Reinforcing the *whole sequence* reinforces whatever made it succeed, and that includes the habit of checking.

The part that should worry you. Nothing in the objective asks the trace to be *true*. It is optimised to precede correct answers, which is not the same thing. We return to this in two slides.

Source: Raschka, *Build Your Own Reasoning Model*, October 2026, citing the DeepSeek-R1 paper.

How self-correction can emerge from outcome rewards

```
Q: Give me two numbers that sum to 10 and multiply to 21.

<think>
Let me try 5 and 5. The sum is 10. The product is 25.
Wait - that is not 21. Let me re-check what I need.
I need a + b = 10 and a * b = 21.
The factor pairs of 21 are (1, 21) and (3, 7).
1 + 21 = 22, no. 3 + 7 = 10, yes.
Check the product: 3 * 7 = 21. Both conditions hold.
</think>

3 and 7.
```

The model starts down a wrong path, **notices**, says so, and recovers. Nobody wrote that behaviour into a dataset and nobody rewarded the recovery. It was reinforced because traces that contain it end in correct answers more often.

Verifiers in practice

Domain	Checker	Where it breaks
mathematics	SymPy or WolframAlpha for symbolic equivalence	extraction and normalisation: is 0.5 the same answer as $\frac{1}{2}$?
code	a compiler, unit tests, or a platform like LeetCode	tests that pass for the wrong reason; timeouts; sandbox escapes
computer use (2026)	success or failure signals from the environment	building the environments in the first place
structured output	a JSON-schema or grammar check	valid syntax, wrong content

The pipeline is always the same three steps: **extract the final answer** $\rightarrow$ **normalise it** $\rightarrow$ **check equivalence**. Most of the engineering is in the first two. The same verifier then does two jobs, scoring the model on your benchmark and supplying the reward during training.

Source: Raschka, *Build Your Own Reasoning Model*, October 2026; *GPT-6 Astra, Looped Transformers, and Hidden Reasoning*.

Which is why GRPO is optional on Thursday

The cost structure. Every step generates G complete responses before a single gradient is computed. Generation dominates, so a GRPO step is one to two orders of magnitude more expensive than an SFT step.

The fragility. Five known corrections, none of them default in every library, and failure looks like a loss curve that is fine while the outputs collapse.

The prerequisite. A verifier you trust, which in practice means an answer extractor you have tested.

Take away

Optional extension B, for GSM8K groups especially: GRPO with an exact-match reward, about 100 steps, as a `gpumedium` batch job.

One hundred steps will not make a reasoning model, but it should show you a reward curve that moves, and that is what you are after here.

Do the main exercise first.

Parameter-efficient training

LoRA

Freeze the original weight matrix and never touch it again. Learn a small correction built from two thin matrices, and add that instead:

$$W' = W + \frac{\alpha}{r} BA$$

In words: a 2560-by-2560 weight matrix holds 6.5 million numbers. Instead of retraining all of them, train a 2560-by-16 matrix and a 16-by-2560 matrix, multiply them and add the result. That is 80 thousand numbers.

A starts random and *B* starts at **zero**, so the correction is nothing at all at step 0 and training begins where the base model was.

Trainable parameters for a 4B model at rank 16, on the attention and MLP matrices: roughly **33M**, or about **0.8%**.

Three knobs

rank — how much the correction can express. 8 to 64 covers almost everything.

alpha — a scale factor. Set it to twice the rank.

target modules — which matrices get an adapter. Attention only is the cheap default; Lab 2 uses all linear layers.

Source: Hu et al. 2021, *LoRA: Low-Rank Adaptation of Large Language Models.*

The memory budget, worked, for a 4B model

Full fine-tuning, mixed precision	
weights, bf16	2 bytes/param
gradients, bf16	2 bytes/param
Adam momentum, fp32	4 bytes/param
Adam variance, fp32	4 bytes/param
fp32 master copy	4 bytes/param
total	**16 bytes/param**

4 billion parameters at 16 bytes each is **64 GB**, *before* a single activation.

LoRA, bf16	
frozen base weights, bf16	8 GB
adapter weights (33M)	0.07 GB
adapter gradients	0.07 GB
Adam state for 33M	0.26 GB
total state	**≈ 8.4 GB**

Plus activations — a few GB at sequence length 1024 with gradient checkpointing. Comfortably under **20 GB**.

One GH200 has **96 GiB**, so full fine-tuning a 4B model would just about fit. LoRA fits five times over, and it trains faster and forgets less.

Merging, and serving adapters

Merge. $W \leftarrow W + \frac{\alpha}{r} BA$ produces an ordinary checkpoint with no adapter and **no inference overhead**.

- merge for a single deployment and for clean benchmarking
- merging is lossy only in the trivial sense that you can no longer take it off
- merge in the *same* precision you trained in, or you will introduce a difference you did not intend

Keep it separate. An adapter is tens of megabytes. Modern servers hot-swap them per request against one shared base model in memory.

- one base model, many tenants, one GPU
- a small amount of extra compute per token
- this is how fine-tuning gets offered as a service

In week 3 you will serve your own adapter this way with vLLM.

For Lab 2: keep the adapter rather than a merged 8 GB checkpoint. It is small and can be loaded again if you reuse it in later labs.

Prompt, retrieve, or fine-tune?

The decision

You want to change	Reach for	Because
behaviour, format, style, tone, output schema, verbosity	**fine-tuning**	gradient descent reshapes the conditional distribution
facts that are private, changing, or need a citation	**retrieval**	weights make a terrible database: no updates, no provenance, no access control
nothing yet; you are still working out the task	**prompting**	no training cost, instant iteration, and it is your baseline anyway

These are not really alternatives. Most production systems are **a fine-tuned model answering over retrieved context**, which is weeks 2 and 5 of this course combined.

What each one cannot fix

Prompting cannot

create a capability the model does not have.

It also cannot make a small model reliable at a long multi-step format. It will comply for six turns and then drift.

Retrieval cannot

fix behaviour. If the model will not follow your output schema, putting more text in its context will not help.

It also cannot fix a model that ignores the context it was given.

Fine-tuning is a poor fit for

changing, attributable facts. It can teach factual associations, but narrow factual training can also **increase** hallucination [Gekhman et al. 2024].

It teaches the model to answer confidently in that shape, which is not the same as knowing.

Use retrieval when facts must be current, attributable or easy to update.

Teams make this mistake every week, and it is an expensive one. Somebody fine-tunes on an internal FAQ, the model learns the *form* of a confident internal answer, and it starts inventing policy numbers in exactly that form.

The order of operations

1. **Write a zero-shot prompt.** Score it on development data. This is your initial baseline.
2. **Improve the prompt.** Few-shot examples, an explicit output format, a system prompt. Score again. Most teams stop improving here long before the prompt is actually good.
3. **Ask what the remaining errors are.** Missing *knowledge* $\rightarrow$ retrieval. Wrong *behaviour* or *format* $\rightarrow$ fine-tuning. Both $\rightarrow$ both.
4. **Lock the protocol, then use the held-out test set once.** Compare the selected prompt and fine-tuned model, including behaviour you were not trying to change.

Take away

Steps 1 and 2 are not a formality. Beating a bad prompt tells you nothing, and beating a good one gives you a result worth writing down.

For your task, does fine-tuning beat your best prompt?

Note everything that question requires of you:

- **your task** — use the task you explored in Lab 1 or choose another suitable task, with its own held-out set
- **your models** — a pretrained base and instruct pair from Hugging Face (Qwen3.5-0.8B by default), not the Week 1 tiny GPT
- **your best prompt** — not your first prompt. Zero-shot and few-shot, on *both* the base and the instruct model
- **beat** — a number on data neither model was trained on, with identical decoding settings on both sides

There is one more, and it is the part most people leave out: **what did it cost?** That is why the table has four numbers rather than three.

Forward pointers: **week 5** is the retrieval branch of the decision slide, **week 6** wraps a tool-using agent around it, and **week 4** is how you find out whether any of these numbers mean anything.

Lab 2 briefing

Material for the final report

Keep one table with **four numbers**, one loss curve and one paragraph.

Setting	Held-out score	Notes
zero-shot, instruct		your floor
best prompt		which model, which prompt, how many shots
LoRA fine-tuned		rank, epochs, learning rate
forgetting check		50 general-knowledge questions, before *and* after

Plus: the training loss curve, and one paragraph answering *was it worth it?*, in GPU-minutes and in points.

A negative result is fine. “Fine-tuning lost to a five-shot prompt, and here is why we think so” scores just as well, provided the numbers are real.

`Week-2/Lab-2/README.md` contains the runnable baseline, LoRA, evaluation and plotting workflow.

The session, and what to do before it

Thursday, 16:15–17:45

0–10	set-up, GPU session, model loaded
10–35	baselines: zero-shot and few-shot, base and instruct
35–70	LoRA SFT
70–90	measure, forgetting check, save results

Thirty-five minutes for the fine-tune is not generous. Arrive with your data already formatted and you should get through it.

No homework compute today or tomorrow

Roihu is down for scheduled maintenance on **3–4 November**. Nothing queues, nothing runs, and the web interface is unavailable.

It returns on **Wednesday evening**. Do the parts that need no GPU now: read your model's chat template, format your `messages` file, write the answer extractor.

Baselines, and three ways to ruin one

The protocol

- zero-shot *and* few-shot
- base *and* instruct
- fixed decoding settings. Greedy removes sampling variation, although exact GPU repeatability is not guaranteed
- the held-out set is **never** touched during development

Choose the best of those four on dev. Report which one it was, then score that locked choice on the held-out test set.

What goes wrong

1. Answer extraction. The model was right and your regular expression disagreed. Read twenty raw generations before you trust any score.

2. Chat template on a base model. It has none. Prompt it as a document, few-shot, with a consistent separator.

3. Contamination. Public benchmark, public model. Go and investigate a suspiciously high zero-shot score before you believe it.

All three move the *baseline*, which is the number everything else gets compared against, so an error here propagates through every remaining lab.

Measuring, optional extensions, and files to keep

The forgetting check Fifty general-knowledge multiple-choice questions, nothing to do with your task. Score the model you started from, score yours, and report the delta, using the same prompt and the same decoding both times.

Optional extensions (Lab 2 README)

A — DPO on a preference set. Start beta at 0.1, and remember that the reference model is just your adapter switched off.

B — GRPO with an exact-match reward, about 100 steps, as a `gpumedium` batch job. GSM8K groups especially.

Keep for the final submission

the results table • the loss curve • the **adapter**, not a merged checkpoint • your config, scripts and Slurm script • your prompt file

Roihu reminders

No compute **3–4 November** — maintenance.

`gputest` before `gpumedium`.

Close idle GPU Jupyter sessions; one forgotten session holds a whole GH200 for twelve hours.

Build environments on the ARM login nodes.

Three things to remember

1. **A chat is a string, and the chat template is the whole interface.** Roles, special tokens and the generation prompt are a format the model was trained to continue.
2. **If you can verify the answer you do not need a reward model.** That observation turned post-training from an annotation budget into a compute budget, and it is what produced reasoning models.
3. **Always measure what fine-tuning broke, not only what it improved.** GLM-5 needed a whole extra distillation stage to repair its own RL stages. Your fifty-question check is the same idea, for one afternoon's work.

Across all three methods, post-training usually reshapes and strengthens behaviour with much less data than pretraining. It can teach task-specific information, but it is not a reliable store for changing facts.

Thursday, and the next two weeks

Thursday 5 November — Lab 2
Does fine-tuning beat your best prompt? Bring formatted data and a working answer extractor.

Week 3 — Aarne Talman, inference
Decoding, KV-cache memory, serving, quantisation and the cost of running your model.

Why inference comes next

Today you changed a model. Next week you put a model behind a server and find out what it costs to run. Reusing today's adapter is useful, but not required.

You will compare the original and FP8 versions with the same held-out metric used today, so speed and quality appear in one table.

Week 4 — Jussi Karlgren, evaluation: metrics and their limits, contamination, LLM-as-a-judge and evaluation across languages.

Reading

The pipeline

- Ouyang et al. 2022, InstructGPT
- Lambert et al. 2024, *Tülu 3*
- Zhou et al. 2023, *LIMA*

Preference optimisation

- Schulman et al. 2017, PPO
- Rafailov et al. 2023, *DPO*
- Ethayarajh et al. 2024, *KTO*
- Hong et al. 2024, *ORPO*
- Meng et al. 2024, *SimPO*

Parameter-efficient training

- Hu et al. 2021, *LoRA*
- Gekhman et al. 2024

RLVR and reasoning

- DeepSeek-AI 2025, *DeepSeek-R1*
- Yu et al. 2025, *DAPO*
- Liu et al. 2025, *Dr. GRPO*
- DeepSeekMath-V2, November 2025

Current practice, Raschka

- *Build Your Own Reasoning Model* (Oct 2026)
- *Controlling Reasoning Effort in LLMs* (Jul 2026)
- *From DeepSeek V3 to V3.2* (Dec 2025)
- *The State of LLMs 2025*

Libraries

- TRL docs: SFT, DPO, KTO, GRPO trainers